BACKEND MODULE DESIGN DOCUMENT

PART-1: BACKEND MODULE ARCHITECTURE

# School ERP

Java 21 + Spring Boot 3 | Spring Data JPA + Hibernate | PostgreSQL + Redis

Prepared by: Backend Architecture Team

Document Version: 1.0

Date: 29 July 2026

# 1. Introduction

This document is the Backend Module Design Document (BMDD) v1.0, Part-1: Backend Module Architecture, for the School ERP System. It is the formal, standalone backend-module reference consolidating the package structure already fixed in Master Development Blueprint (MDB) Part-1, the layer standards already fixed in MDB Part-3, and the DTO-related response conventions already fixed in the API Specification Document (ASD) Parts 1-3, now organized specifically around backend module design: structure, layering, dependency rules, DTOs, mapping, and inter-module communication.

This document does not regenerate RGD v2.0, any prior Appendix, any UI/UX Screen Specification, any MDB Part, any Database Design Document (DDD) Part, or any ASD Part. It contains no source code, no Java classes, no package implementation, no controller/service/repository/entity code, no UML, no sequence diagrams, and no implementation examples — it fixes structure and convention, not a concrete codebase.

Where a standard is already fixed elsewhere (module list and package shape in MDB Part-1 Section 6, layer boundaries in MDB Part-3 Sections 23-24, DTO/response shape in ASD Part-1 Section 10), this document consolidates it at the level of detail a backend developer needs for day-to-day module design, without contradiction or re-derivation.

## 2. Backend Architecture Overview

Confirms, without altering, the backend's position within the system architecture already fixed in MDB Part-1 Section 4 and Appendix-I Section 7.

- The backend is a single Spring Boot 3 application, horizontally scaled across multiple stateless instances (MDB Part-1 Section 4), serving every API Group already catalogued in Appendix-K Section 18 through the layered Controller-Service-Repository structure already fixed in Appendix-J Section 6.
- The backend is organized as 11 module packages plus one shared administration package (MDB Part-1 Section 6) — this module list is fixed and matches the 11 business modules already established throughout RGD v2.0 and the UI/UX Screen Specifications exactly, with no additional or renamed module introduced at the backend-architecture level.
- PostgreSQL (via Spring Data JPA/Hibernate, DDD Parts 1-3) and Redis (MDB Part-3 Section 26) are the backend's only persistence and caching dependencies — no additional data store is introduced without a formal revision to this document and the DDD.
- The backend exposes no interface to the React + TypeScript frontend other than the API already fully specified in ASD Parts 1-4 — this BMDD does not introduce an alternate integration surface.

## 3. Module Structure Standards

Fixes the internal shape every one of the 11 business modules plus the shared administration module must follow, extending MDB Part-1 Section 6.

| Module | Governing FR/BR Range (Reused) |
|---|---|
| Admission | FR-01 to FR-05c, BR-ADM-001 to BR-ADM-008 |
| Student Information | FR-06 to FR-09, BR-SIS-001 to BR-SIS-006 |
| Attendance | FR-10 to FR-13, BR-ATT-001 to BR-ATT-007 |
| Examination | FR-17 to FR-21, BR-EXM-001 to BR-EXM-007 |
| Fees | FR-22 to FR-26b, BR-FEE-001 to BR-FEE-008 |
| Library | FR-27 to FR-29, BR-LIB-001 to BR-LIB-006 |
| Transport | FR-30 to FR-32, BR-TRN-001 to BR-TRN-006 |
| HR & Payroll | FR-33 to FR-36, BR-HR-001 to BR-HR-007 |
| Communication | FR-37 to FR-39, BR-COM-001 to BR-COM-005 |
| Reports | FR-40 to FR-42, BR-RPT-001 to BR-RPT-005 |
| Administration (shared) | Cross-cutting; no dedicated FR range of its own |

Every module package contains, at minimum, the five sub-packages already fixed in MDB Part-1 Section 6 (controller, service, repository, dto, entity) plus a mapper sub-package (Section 4/8 this document) — no module is permitted a structurally different internal shape from any other.

## 4. Package Structure Standards

Extends MDB Part-1 Section 6 with the addition of a dedicated mapper sub-package, formalizing where entity-to-DTO conversion (Section 8 this document) resides.

| Sub-Package | Contents |
|---|---|
| controller | Controller components only (Section 5) — no business logic |
| service | Service components only (Section 5) — all Business Rule enforcement |
| repository | Repository interfaces only (Section 5) — data access, no business logic |
| dto | Request and Response DTOs (Section 7) for this module's API Groups |
| entity | JPA entity classes mapping this module's tables (DDD Parts 1-3) |
| mapper | Entity-to-DTO and DTO-to-entity conversion components (Section 8) |

This six-sub-package shape is uniform across all 11 business modules and the administration module — a module introducing a seventh sub-package, or omitting one of these six, requires an explicit, reviewed exception, not a silent per-module variation.

## 5. Layer Responsibilities

Consolidates the layer boundaries already fixed in Appendix-J Sections 5-6 and MDB Part-3 Sections 23-24 into a single definitive reference.

| Layer | Responsibility | Explicitly Excluded |
|---|---|---|
| Controller | Receive the API request (ASD Part-1), delegate to exactly one Service method, return its result | Business logic, validation beyond basic request-shape checks, direct Repository access |
| Service | Enforce Business Rules (Appendix-C v1.1), orchestrate Repository calls, call AuditLogService, raise domain events (Section 9) | Direct database access without going through a Repository; direct DTO serialization for the response |
| Repository | Data access via Spring Data JPA (DDD Part-3) | Business logic, validation, cross-repository orchestration |
| Mapper | Convert between entity and DTO (Section 8) | Business logic, validation, data access |
| Entity | Represent a persisted row (DDD Parts 1-3) | Any behavior beyond simple, persistence-related accessors |

## 6. Dependency Rules

Fixes allowable dependencies between packages, extending MDB Part-1 Section 8 and MDB Part-3 Section 24.

- A module's Service package may depend on another module's published Service interface, but never on another module's Repository or Entity package directly — this is the single most important rule in this document and is never relaxed for convenience.
- A module's Controller package depends only on its own module's Service package — a Controller never calls another module's Service directly, preserving the one-Controller-to-one-module mapping already fixed in Section 3.
- The administration package (shared services: AuditLogService, ConfigurationService, ApprovalRequestService, DocumentService, NotificationService, UserService, RoleService — MDB Part-1 Section 6) is the only package every other module is permitted to depend on directly, consistent with its shared-service role.
- Cross-module effects not expressible as a direct Service-interface dependency (Library fine posting to Fees, Transport fee linkage, Communication notification triggers) are implemented exclusively through the Event Publisher/Consumer pattern (Section 9), never as a disguised direct dependency.
- Dependency direction never cycles — if Module A's Service depends on Module B's Service interface, Module B's Service never depends back on Module A's Service interface; a genuine bidirectional need is resolved through the event pattern (Section 9) instead.

## 7. DTO Standards

Fixes DTO design conventions, extending ASD Part-1 Section 10's "response bodies are always DTOs" rule and MDB Part-3 Section 24.

- A Request DTO and a Response DTO for the same logical resource are distinct types, even when their field sets overlap heavily — a single DTO is never reused for both directions, since a Request DTO's mandatory/optional shape (matching the UI/UX Screen Specifications' Form Fields) often differs from a Response DTO's fully-populated shape.
- Every Request DTO carries its Bean Validation constraints (MDB Part-2 Section 16) directly on its fields — validation annotations live on the DTO, not duplicated separately in the Controller or Service.
- A DTO never exposes a Sensitive/PII field to a context where the Role Permission model (ASD Part-2 Section 12) would not permit it — where a field's visibility is role-dependent, the module provides a role-scoped DTO variant rather than a single DTO with runtime field suppression logic scattered through Service code.
- A DTO is immutable once constructed wherever the chosen DTO implementation pattern supports it, avoiding accidental, hard-to-trace mutation of request data as it passes from Controller to Service to Mapper.

## 8. Mapper Standards

Fixes the conversion boundary between entity and DTO, extending MDB Part-1 Section 10.1 and MDB Part-3 Section 24's mapping-at-the-Service-boundary rule.

- Every module's mapper sub-package (Section 4) contains exactly the mapping components needed to convert between that module's entities and DTOs — a mapper never performs business logic, validation, or data access; it is a pure structural transformation.
- Entity-to-DTO and DTO-to-entity conversion occurs only at the Service layer boundary (MDB Part-3 Section 24) — a Controller never constructs or invokes a mapper directly, and a Repository never returns a DTO.
- The specific mapping mechanism (a mapping-generation framework versus hand-written mapper methods) remains Client/Product Decision Required — whichever approach is chosen must produce mapper components confined to the mapper sub-package, with no mapping logic duplicated inline inside Service methods.
- A mapper never silently drops a Sensitive/PII field inconsistently across call sites — role-scoped field suppression (Section 7) is handled by selecting the appropriate DTO variant before mapping, not by conditional logic inside the mapper itself.

## 9. Module Communication Standards

Fixes the Event Publisher/Consumer pattern already referenced in Appendix-J Section 5 and MDB Part-3 Section 24 as the mechanism for indirect cross-module communication.

- A module raises a domain event (e.g., "Book Returned Late," "Transport Allocation Confirmed," "Exam Result Published") from its Service layer after its own transaction (DDD Part-3 Section 21 equivalent) completes successfully — an event is never raised speculatively before the originating change is committed.
- A consuming module's Service layer subscribes to the specific events it needs (e.g., Fees consuming a Library fine event) without depending on the publishing module's Service interface directly — this preserves the no-direct-dependency rule (Section 6) for effects that are conceptually cross-module rather than a direct data need.
- Event delivery failure (a consumer temporarily unavailable) is handled via retry, consistent with the Background Job/retry pattern already fixed in Appendix-J Section 17 — an event is not silently dropped on first-attempt failure.
- Direct Service-interface dependency (Section 6) remains preferred over the event pattern wherever a synchronous, immediate result is genuinely required (e.g., Fee Collection needing an immediate Student record lookup) — the event pattern is reserved for effects that are inherently asynchronous or eventually-consistent by business nature, not used as a default for all cross-module interaction.

## 10. Backend Review Checklist

Verified for every new or changed backend module component, complementing the general Code Review Checklist already fixed in MDB Part-3 Section 30 with module-architecture-specific items.

| # | Checklist Item |
|---|---|
| 1 | New/changed code resides in the correct module package (Section 3) matching its governing FR/BR range |
| 2 | The six-sub-package shape (Section 4) is intact — no ad hoc additional sub-package introduced without exception review |
| 3 | Layer responsibilities (Section 5) are respected — no business logic in Controller/Repository/Mapper/Entity |
| 4 | No Service depends on another module's Repository or Entity directly (Section 6) |
| 5 | Request and Response DTOs are distinct types with correct validation and role-scoped field visibility (Section 7) |
| 6 | Mapping logic resides only in the mapper sub-package, invoked only from the Service layer (Section 8) |
| 7 | Cross-module effects not expressible as a direct Service dependency use the Event Publisher/Consumer pattern (Section 9) |
| 8 | No circular dependency introduced between any two modules' Service packages (Section 6) |